

"AgriConnect - Offline-First Farmer Marketplace & Advisory Platform"

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

K. Vaishnavi: 2520030198

M. Likhitha: 2520030575

S. Nikhil: 2520030246

Under the Esteemed Guidance of

Dr Spandana.S

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled "AgriConnect - Offline-First Farmer Marketplace & Advisory Platform" is a record of bonafide work of K. Vaishnavi - 2520030198, M. Likhitha - 2520030575, and S. Nikhil - 2520030246 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The report presents the project concept, system requirements, technology stack, design, implementation approach, features, testing approach, deployment considerations, future scope, and conclusion based on the project review material.

K. Vaishnavi: 2520030198

M. Likhitha: 2520030575

S. Nikhil: 2520030246

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled "AgriConnect - Offline-First Farmer Marketplace & Advisory Platform" is a bonafide work done and submitted by K. Vaishnavi - 2520030198, M. Likhitha - 2520030575, and S. Nikhil - 2520030246 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science and Engineering, K L (Deemed to be) University.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who have assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Spandana.S**, and my guide **Dr Spandana.S**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to Dr. P Venkateshwara Roa (CSE), Head of the **Department for Computer Science Engineering**, for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

K. Vaishnavi: 2520030198

M. Likhitha: 2520030575

S. Nikhil: 2520030246

Abstract

AgriConnect is an offline-first farmer marketplace and agricultural advisory platform designed to support farmers in areas with poor or unreliable internet connectivity. It provides offline access to complete crop information, including planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance.

The platform also enables farmers to manage products, interact with buyers, view buyer trust ratings, and manage orders. Market prices and marketplace data are synchronized and updated whenever internet connectivity is available, while essential information remains accessible offline.

AgriConnect uses HTML, CSS, and JavaScript for the frontend, Java and Spring Boot for the backend, MySQL for the database, Postman for API testing, IntelliJ IDEA or VS Code for development, Git and GitHub for version control, Maven as the build tool, and Local Storage or Indexed DB for offline storage.

Index

1	Introduction: Background, Problem statement, Scope, Objectives, Importance, Target users	5-15
2	System Requirements: Hardware, Software, Development tools	16-21
3	Technology Stack: Front-end, Back-end, Database, Version Control, APIs	22-32
4	System Architecture: High-level architecture, Layers, Deployment	33-36
5	Design: ER Diagram / Database schema	37
6	Implementation: Modules, Front-end logic, API integration, Authentication	38-47
7	Features: Main features and user/technical descriptions	48-55
8	Testing: Postman, Integration, Tools, Test cases and results	56-64
9	Deployment: Steps, Environment, Hosting	65-73
10	Challenges & Limitations: Issues, Solutions, Current limitations	74-82
11	Future Enhancements: Planned features, Integrations and optimizations	83-88
12	Conclusion: Summary, Achievements, Skills learned	89-97

13	References: Books, tutorials, documentation	98-99
14	Appendices: Screenshots, Code, Setup, User guide, Geo-tag, Review forms	100-104

CHAPTER -1 INTRODUCTION

1.5 Importance of the Application

The project is important because it connects two needs: agricultural knowledge and market participation. A farmer should be able to consult useful crop information and continue important marketplace activities even when the network is unreliable.

The target users are farmers who work in rural or remote areas and may experience low or intermittent connectivity. Buyers are also part of the marketplace because farmers need a practical way to interact with buyers and manage products and orders.

Agriculture plays an important role in the livelihood of millions of people in India. Farmers depend on timely information about crops, cultivation activities, market conditions, and selling opportunities.

Digital platforms can support these activities, but their usefulness is reduced when internet connectivity is weak or unavailable.

AgriConnect is designed as an offline-first farmer marketplace and agricultural advisory platform. The system is intended to work in no-network and low-network conditions while keeping important agricultural guidance accessible locally.

The offline-first approach is the defining requirement of the platform. Essential knowledge remains accessible locally, while dynamic marketplace information is refreshed after connectivity is restored.

1.6 Target Users / Audience

Agriculture plays an important role in the livelihood of millions of people in India. Farmers depend on timely information about crops, cultivation activities, market conditions, and selling opportunities.

Digital platforms can support these activities, but their usefulness is reduced when internet connectivity is weak or unavailable.

AgriConnect is designed as an offline-first farmer marketplace and agricultural advisory platform. The system is intended to work in no-network and low-network conditions while keeping important agricultural guidance accessible locally.

The advisory component provides information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. The marketplace component supports product management, buyer interaction, buyer trust ratings, and order management.

Dynamic information such as market prices is different from static or reusable crop guidance. AgriConnect therefore synchronizes changing market and marketplace information whenever connectivity becomes available while keeping essential information available offline.

The offline-first approach is the defining requirement of the platform. Essential knowledge remains accessible locally, while dynamic marketplace information is refreshed after connectivity is restored.

1.1 Background of the Project

The advisory component provides information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. The marketplace component supports product management, buyer interaction, buyer trust ratings, and order management.

Dynamic information such as market prices is different from static or reusable crop guidance. AgriConnect therefore synchronizes changing market and marketplace information whenever connectivity becomes available while keeping essential information available offline.

The project is important because it connects two needs: agricultural knowledge and market participation. A farmer should be able to consult useful crop information and continue important marketplace activities even when the network is unreliable.

The target users are farmers who work in rural or remote areas and may experience low or intermittent connectivity. Buyers are also part of the marketplace because farmers need a practical way to interact with buyers and manage products and orders.

The offline-first approach is the defining requirement of the platform. Essential knowledge remains accessible locally, while dynamic marketplace information is refreshed after connectivity is restored.

1.2 Problem Statement

The project is important because it connects two needs: agricultural knowledge and market participation. A farmer should be able to consult useful crop information and continue important marketplace activities even when the network is unreliable.

The target users are farmers who work in rural or remote areas and may experience low or intermittent connectivity. Buyers are also part of the marketplace because farmers need a practical way to interact with buyers and manage products and orders.

Agriculture plays an important role in the livelihood of millions of people in India. Farmers depend on timely information about crops, cultivation activities, market conditions, and selling opportunities.

Digital platforms can support these activities, but their usefulness is reduced when internet connectivity is weak or unavailable.

AgriConnect is designed as an offline-first farmer marketplace and agricultural advisory platform. The system is intended to work in no-network and low-network conditions while keeping important agricultural guidance accessible locally.

The offline-first approach is the defining requirement of the platform. Essential knowledge remains accessible locally, while dynamic marketplace information is refreshed after connectivity is restored.

1.3 Scope of the Project

Agriculture plays an important role in the livelihood of millions of people in India. Farmers depend on timely information about crops, cultivation activities, market conditions, and selling opportunities.

Digital platforms can support these activities, but their usefulness is reduced when internet connectivity is weak or unavailable.

AgriConnect is designed as an offline-first farmer marketplace and agricultural advisory platform. The system is intended to work in no-network and low-network conditions while keeping important agricultural guidance accessible locally.

The advisory component provides information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. The marketplace component supports product management, buyer interaction, buyer trust ratings, and order management.

Dynamic information such as market prices is different from static or reusable crop guidance. AgriConnect therefore synchronizes changing market and marketplace information whenever connectivity becomes available while keeping essential information available offline.

The offline-first approach is the defining requirement of the platform. Essential knowledge remains accessible locally, while dynamic marketplace information is refreshed after connectivity is restored.

1.4 Objectives

The advisory component provides information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. The marketplace component supports product management, buyer interaction, buyer trust ratings, and order management.

Dynamic information such as market prices is different from static or reusable crop guidance. AgriConnect therefore synchronizes changing market and marketplace information whenever connectivity becomes available while keeping essential information available offline.

The project is important because it connects two needs: agricultural knowledge and market participation. A farmer should be able to consult useful crop information and continue important marketplace activities even when the network is unreliable.

The target users are farmers who work in rural or remote areas and may experience low or intermittent connectivity. Buyers are also part of the marketplace because farmers need a practical way to interact with buyers and manage products and orders.

The offline-first approach is the defining requirement of the platform. Essential knowledge remains accessible locally, while dynamic marketplace information is refreshed after connectivity is restored.

1.5 Importance of the Application

The project is important because it connects two needs: agricultural knowledge and market participation. A farmer should be able to consult useful crop information and continue important marketplace activities even when the network is unreliable.

The target users are farmers who work in rural or remote areas and may experience low or intermittent connectivity. Buyers are also part of the marketplace because farmers need a practical way to interact with buyers and manage products and orders.

Agriculture plays an important role in the livelihood of millions of people in India. Farmers depend on timely information about crops, cultivation activities, market conditions, and selling opportunities.

Digital platforms can support these activities, but their usefulness is reduced when internet connectivity is weak or unavailable.

AgriConnect is designed as an offline-first farmer marketplace and agricultural advisory platform. The system is intended to work in no-network and low-network conditions while keeping important agricultural guidance accessible locally.

The offline-first approach is the defining requirement of the platform. Essential knowledge remains accessible locally, while dynamic marketplace information is refreshed after connectivity is restored.

1.6 Target Users / Audience

Agriculture plays an important role in the livelihood of millions of people in India. Farmers depend on timely information about crops, cultivation activities, market conditions, and selling opportunities. Digital platforms can support these activities, but their usefulness is reduced when internet connectivity is weak or unavailable.

AgriConnect is designed as an offline-first farmer marketplace and agricultural advisory platform. The system is intended to work in no-network and low-network conditions while keeping important agricultural guidance accessible locally.

The advisory component provides information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. The marketplace component supports product management, buyer interaction, buyer trust ratings, and order management.

Dynamic information such as market prices is different from static or reusable crop guidance. AgriConnect therefore synchronizes changing market and marketplace information whenever connectivity becomes available while keeping essential information available offline.

The offline-first approach is the defining requirement of the platform. Essential knowledge remains accessible locally, while dynamic marketplace information is refreshed after connectivity is restored.

1.1 Background of the Project

The advisory component provides information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. The marketplace component supports product management, buyer interaction, buyer trust ratings, and order management.

Dynamic information such as market prices is different from static or reusable crop guidance. AgriConnect therefore synchronizes changing market and marketplace information whenever connectivity becomes available while keeping essential information available offline.

The project is important because it connects two needs: agricultural knowledge and market participation. A farmer should be able to consult useful crop information and continue important marketplace activities even when the network is unreliable.

The target users are farmers who work in rural or remote areas and may experience low or intermittent connectivity. Buyers are also part of the marketplace because farmers need a practical way to interact with buyers and manage products and orders.

The offline-first approach is the defining requirement of the platform. Essential knowledge remains accessible locally, while dynamic marketplace information is refreshed after connectivity is restored.

1.2 Problem Statement

The project is important because it connects two needs: agricultural knowledge and market participation. A farmer should be able to consult useful crop information and continue important marketplace activities even when the network is unreliable.

The target users are farmers who work in rural or remote areas and may experience low or intermittent connectivity. Buyers are also part of the marketplace because farmers need a practical way to interact with buyers and manage products and orders.

Agriculture plays an important role in the livelihood of millions of people in India. Farmers depend on timely information about crops, cultivation activities, market conditions, and selling opportunities.

Digital platforms can support these activities, but their usefulness is reduced when internet connectivity is weak or unavailable.

AgriConnect is designed as an offline-first farmer marketplace and agricultural advisory platform. The system is intended to work in no-network and low-network conditions while keeping important agricultural guidance accessible locally.

The offline-first approach is the defining requirement of the platform. Essential knowledge remains accessible locally, while dynamic marketplace information is refreshed after connectivity is restored.

1.3 Scope of the Project

Agriculture plays an important role in the livelihood of millions of people in India. Farmers depend on timely information about crops, cultivation activities, market conditions, and selling opportunities.

Digital platforms can support these activities, but their usefulness is reduced when internet connectivity is weak or unavailable.

AgriConnect is designed as an offline-first farmer marketplace and agricultural advisory platform. The system is intended to work in no-network and low-network conditions while keeping important agricultural guidance accessible locally.

The advisory component provides information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. The marketplace component supports product management, buyer interaction, buyer trust ratings, and order management.

Dynamic information such as market prices is different from static or reusable crop guidance. AgriConnect therefore synchronizes changing market and marketplace information whenever connectivity becomes available while keeping essential information available offline.

The offline-first approach is the defining requirement of the platform. Essential knowledge remains accessible locally, while dynamic marketplace information is refreshed after connectivity is restored.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The hardware requirements for AgriConnect are intended to support normal web application development and use. A computer with a modern processor, sufficient memory, storage, and network capability can be used for development and testing. Farmer-side use can be through a compatible device capable of running a modern browser and local storage.

The software environment includes Windows or Linux as the operating system. The frontend uses HTML, CSS, and JavaScript. The backend uses Java with Spring Boot, and MySQL is used as the database. These choices are directly specified in the project review material.

Postman is used for API testing. IntelliJ IDEA or VS Code can be used as the development environment. Git and GitHub provide version control, while Maven is used as the build tool for the Java and Spring Boot backend.

Local Storage or Indexed DB provides the proposed offline storage layer. It allows information and offline actions to remain available on the device until synchronization can occur. This supports the central offline-first requirement of the project.

The specified environment is Windows/Linux with HTML, CSS, JavaScript, Java, Spring Boot, MySQL, Postman, IntelliJ IDEA or VS Code, Git, GitHub, Maven, and Local Storage or Indexed DB.

2.2 Software Requirements

Postman is used for API testing. IntelliJ IDEA or VS Code can be used as the development environment. Git and GitHub provide version control, while Maven is used as the build tool for the Java and Spring Boot backend.

Local Storage or Indexed DB provides the proposed offline storage layer. It allows information and offline actions to remain available on the device until synchronization can occur. This supports the central offline-first requirement of the project.

The hardware requirements for AgriConnect are intended to support normal web application development and use. A computer with a modern processor, sufficient memory, storage, and network capability can be used for development and testing. Farmer-side use can be through a compatible device capable of running a modern browser and local storage.

The software environment includes Windows or Linux as the operating system. The frontend uses HTML, CSS, and JavaScript. The backend uses Java with Spring Boot, and MySQL is used as the database. These choices are directly specified in the project review material.

The specified environment is Windows/Linux with HTML, CSS, JavaScript, Java, Spring Boot, MySQL, Postman, IntelliJ IDEA or VS Code, Git, GitHub, Maven, and Local Storage or Indexed DB.

2.3 Development Tools and Frameworks

The hardware requirements for AgriConnect are intended to support normal web application development and use. A computer with a modern processor, sufficient memory, storage, and network capability can be used for development and testing. Farmer-side use can be through a compatible device capable of running a modern browser and local storage.

The software environment includes Windows or Linux as the operating system. The frontend uses HTML, CSS, and JavaScript. The backend uses Java with Spring Boot, and MySQL is used as the database. These choices are directly specified in the project review material.

Postman is used for API testing. IntelliJ IDEA or VS Code can be used as the development environment. Git and GitHub provide version control, while Maven is used as the build tool for the Java and Spring Boot backend.

Local Storage or Indexed DB provides the proposed offline storage layer. It allows information and offline actions to remain available on the device until synchronization can occur. This supports the central offline-first requirement of the project.

The specified environment is Windows/Linux with HTML, CSS, JavaScript, Java, Spring Boot, MySQL, Postman, IntelliJ IDEA or VS Code, Git, GitHub, Maven, and Local Storage or Indexed DB.

2.1 Hardware Requirements

Postman is used for API testing. IntelliJ IDEA or VS Code can be used as the development environment. Git and GitHub provide version control, while Maven is used as the build tool for the Java and Spring Boot backend.

Local Storage or Indexed DB provides the proposed offline storage layer. It allows information and offline actions to remain available on the device until synchronization can occur. This supports the central offline-first requirement of the project.

The hardware requirements for AgriConnect are intended to support normal web application development and use. A computer with a modern processor, sufficient memory, storage, and network capability can be used for development and testing. Farmer-side use can be through a compatible device capable of running a modern browser and local storage.

The software environment includes Windows or Linux as the operating system. The frontend uses HTML, CSS, and JavaScript. The backend uses Java with Spring Boot, and MySQL is used as the database. These choices are directly specified in the project review material.

The specified environment is Windows/Linux with HTML, CSS, JavaScript, Java, Spring Boot, MySQL, Postman, IntelliJ IDEA or VS Code, Git, GitHub, Maven, and Local Storage or Indexed DB.

2.2 Software Requirements

The hardware requirements for AgriConnect are intended to support normal web application development and use. A computer with a modern processor, sufficient memory, storage, and network capability can be used for development and testing. Farmer-side use can be through a compatible device capable of running a modern browser and local storage.

The software environment includes Windows or Linux as the operating system. The frontend uses HTML, CSS, and JavaScript. The backend uses Java with Spring Boot, and MySQL is used as the database. These choices are directly specified in the project review material.

Postman is used for API testing. IntelliJ IDEA or VS Code can be used as the development environment. Git and GitHub provide version control, while Maven is used as the build tool for the Java and Spring Boot backend.

Local Storage or Indexed DB provides the proposed offline storage layer. It allows information and offline actions to remain available on the device until synchronization can occur. This supports the central offline-first requirement of the project.

The specified environment is Windows/Linux with HTML, CSS, JavaScript, Java, Spring Boot, MySQL, Postman, IntelliJ IDEA or VS Code, Git, GitHub, Maven, and Local Storage or Indexed DB.

2.3 Development Tools and Frameworks

Postman is used for API testing. IntelliJ IDEA or VS Code can be used as the development environment. Git and GitHub provide version control, while Maven is used as the build tool for the Java and Spring Boot backend.

Local Storage or Indexed DB provides the proposed offline storage layer. It allows information and offline actions to remain available on the device until synchronization can occur. This supports the central offline-first requirement of the project.

The hardware requirements for AgriConnect are intended to support normal web application development and use. A computer with a modern processor, sufficient memory, storage, and network capability can be used for development and testing. Farmer-side use can be through a compatible device capable of running a modern browser and local storage.

The software environment includes Windows or Linux as the operating system. The frontend uses HTML, CSS, and JavaScript. The backend uses Java with Spring Boot, and MySQL is used as the database. These choices are directly specified in the project review material.

The specified environment is Windows/Linux with HTML, CSS, JavaScript, Java, Spring Boot, MySQL, Postman, IntelliJ IDEA or VS Code, Git, GitHub, Maven, and Local Storage or Indexed DB.

CHAPTER-3 TECHNOLOGY STACK

3.2 Back-End

Local Storage or Indexed DB is used as the offline storage layer. The technology stack therefore separates online persistence in MySQL from local device storage used to maintain continuity during connectivity interruptions.

The frontend of AgriConnect uses HTML, CSS, and JavaScript. The interface is responsible for presenting crop information, product information, buyer details, trust ratings, orders, and market information in a form that farmers can use easily.

The backend is based on Java and Spring Boot. It provides the application services and APIs required for marketplace operations, synchronization, and access to persistent data. The backend acts as the central online component when connectivity is available.

MySQL is the database technology specified for the project. Structured information such as farmer profiles, products, buyers, orders, market information, and synchronization records can be represented in relational tables.

The stack separates the presentation layer, Spring Boot application services, MySQL persistence, and local offline storage. This separation supports maintainability and the synchronization requirement.

3.3 Database

The backend is based on Java and Spring Boot. It provides the application services and APIs required for marketplace operations, synchronization, and access to persistent data. The backend acts as the central online component when connectivity is available.

MySQL is the database technology specified for the project. Structured information such as farmer profiles, products, buyers, orders, market information, and synchronization records can be represented in relational tables.

Git and GitHub are used for version control so that changes to the project can be tracked. Maven is used as the build tool for the Java and Spring Boot backend. Postman supports API testing during development.

Local Storage or Indexed DB is used as the offline storage layer. The technology stack therefore separates online persistence in MySQL from local device storage used to maintain continuity during connectivity interruptions.

The stack separates the presentation layer, Spring Boot application services, MySQL persistence, and local offline storage. This separation supports maintainability and the synchronization requirement.

3.4 Version Control

Git and GitHub are used for version control so that changes to the project can be tracked. Maven is used as the build tool for the Java and Spring Boot backend. Postman supports API testing during development.

Local Storage or Indexed DB is used as the offline storage layer. The technology stack therefore separates online persistence in MySQL from local device storage used to maintain continuity during connectivity interruptions.

The frontend of AgriConnect uses HTML, CSS, and JavaScript. The interface is responsible for presenting crop information, product information, buyer details, trust ratings, orders, and market information in a form that farmers can use easily.

The backend is based on Java and Spring Boot. It provides the application services and APIs required for marketplace operations, synchronization, and access to persistent data. The backend acts as the central online component when connectivity is available.

The stack separates the presentation layer, Spring Boot application services, MySQL persistence, and local offline storage. This separation supports maintainability and the synchronization requirement.

3.5 APIs and Supporting Services

The frontend of AgriConnect uses HTML, CSS, and JavaScript. The interface is responsible for presenting crop information, product information, buyer details, trust ratings, orders, and market information in a form that farmers can use easily.

The backend is based on Java and Spring Boot. It provides the application services and APIs required for marketplace operations, synchronization, and access to persistent data. The backend acts as the central online component when connectivity is available.

MySQL is the database technology specified for the project. Structured information such as farmer profiles, products, buyers, orders, market information, and synchronization records can be represented in relational tables.

Git and GitHub are used for version control so that changes to the project can be tracked. Maven is used as the build tool for the Java and Spring Boot backend. Postman supports API testing during development.

The stack separates the presentation layer, Spring Boot application services, MySQL persistence, and local offline storage. This separation supports maintainability and the synchronization requirement.

3.1 Front-End

MySQL is the database technology specified for the project. Structured information such as farmer profiles, products, buyers, orders, market information, and synchronization records can be represented in relational tables.

Git and GitHub are used for version control so that changes to the project can be tracked. Maven is used as the build tool for the Java and Spring Boot backend. Postman supports API testing during development.

Local Storage or Indexed DB is used as the offline storage layer. The technology stack therefore separates online persistence in MySQL from local device storage used to maintain continuity during connectivity interruptions.

The frontend of AgriConnect uses HTML, CSS, and JavaScript. The interface is responsible for presenting crop information, product information, buyer details, trust ratings, orders, and market information in a form that farmers can use easily.

The stack separates the presentation layer, Spring Boot application services, MySQL persistence, and local offline storage. This separation supports maintainability and the synchronization requirement.

3.2 Back-End

Local Storage or Indexed DB is used as the offline storage layer. The technology stack therefore separates online persistence in MySQL from local device storage used to maintain continuity during connectivity interruptions.

The frontend of AgriConnect uses HTML, CSS, and JavaScript. The interface is responsible for presenting crop information, product information, buyer details, trust ratings, orders, and market information in a form that farmers can use easily.

The backend is based on Java and Spring Boot. It provides the application services and APIs required for marketplace operations, synchronization, and access to persistent data. The backend acts as the central online component when connectivity is available.

MySQL is the database technology specified for the project. Structured information such as farmer profiles, products, buyers, orders, market information, and synchronization records can be represented in relational tables.

The stack separates the presentation layer, Spring Boot application services, MySQL persistence, and local offline storage. This separation supports maintainability and the synchronization requirement.

3.3 Database

The backend is based on Java and Spring Boot. It provides the application services and APIs required for marketplace operations, synchronization, and access to persistent data. The backend acts as the central online component when connectivity is available.

MySQL is the database technology specified for the project. Structured information such as farmer profiles, products, buyers, orders, market information, and synchronization records can be represented in relational tables.

Git and GitHub are used for version control so that changes to the project can be tracked. Maven is used as the build tool for the Java and Spring Boot backend. Postman supports API testing during development.

Local Storage or Indexed DB is used as the offline storage layer. The technology stack therefore separates online persistence in MySQL from local device storage used to maintain continuity during connectivity interruptions.

The stack separates the presentation layer, Spring Boot application services, MySQL persistence, and local offline storage. This separation supports maintainability and the synchronization requirement.

3.4 Version Control

Git and GitHub are used for version control so that changes to the project can be tracked. Maven is used as the build tool for the Java and Spring Boot backend. Postman supports API testing during development.

Local Storage or Indexed DB is used as the offline storage layer. The technology stack therefore separates online persistence in MySQL from local device storage used to maintain continuity during connectivity interruptions.

The frontend of AgriConnect uses HTML, CSS, and JavaScript. The interface is responsible for presenting crop information, product information, buyer details, trust ratings, orders, and market information in a form that farmers can use easily.

The backend is based on Java and Spring Boot. It provides the application services and APIs required for marketplace operations, synchronization, and access to persistent data. The backend acts as the central online component when connectivity is available.

The stack separates the presentation layer, Spring Boot application services, MySQL persistence, and local offline storage. This separation supports maintainability and the synchronization requirement.

3.5 APIs and Supporting Services

The frontend of AgriConnect uses HTML, CSS, and JavaScript. The interface is responsible for presenting crop information, product information, buyer details, trust ratings, orders, and market information in a form that farmers can use easily.

The backend is based on Java and Spring Boot. It provides the application services and APIs required for marketplace operations, synchronization, and access to persistent data. The backend acts as the central online component when connectivity is available.

MySQL is the database technology specified for the project. Structured information such as farmer profiles, products, buyers, orders, market information, and synchronization records can be represented in relational tables.

Git and GitHub are used for version control so that changes to the project can be tracked. Maven is used as the build tool for the Java and Spring Boot backend. Postman supports API testing during development.

The stack separates the presentation layer, Spring Boot application services, MySQL persistence, and local offline storage. This separation supports maintainability and the synchronization requirement.

3.1 Front-End

MySQL is the database technology specified for the project. Structured information such as farmer profiles, products, buyers, orders, market information, and synchronization records can be represented in relational tables.

Git and GitHub are used for version control so that changes to the project can be tracked. Maven is used as the build tool for the Java and Spring Boot backend. Postman supports API testing during development.

Local Storage or Indexed DB is used as the offline storage layer. The technology stack therefore separates online persistence in MySQL from local device storage used to maintain continuity during connectivity interruptions.

The frontend of AgriConnect uses HTML, CSS, and JavaScript. The interface is responsible for presenting crop information, product information, buyer details, trust ratings, orders, and market information in a form that farmers can use easily.

The stack separates the presentation layer, Spring Boot application services, MySQL persistence, and local offline storage. This separation supports maintainability and the synchronization requirement.

3.2 Back-End

Local Storage or Indexed DB is used as the offline storage layer. The technology stack therefore separates online persistence in MySQL from local device storage used to maintain continuity during connectivity interruptions.

The frontend of AgriConnect uses HTML, CSS, and JavaScript. The interface is responsible for presenting crop information, product information, buyer details, trust ratings, orders, and market information in a form that farmers can use easily.

The backend is based on Java and Spring Boot. It provides the application services and APIs required for marketplace operations, synchronization, and access to persistent data. The backend acts as the central online component when connectivity is available.

MySQL is the database technology specified for the project. Structured information such as farmer profiles, products, buyers, orders, market information, and synchronization records can be represented in relational tables.

The stack separates the presentation layer, Spring Boot application services, MySQL persistence, and local offline storage. This separation supports maintainability and the synchronization requirement.

CHAPTER -4 SYSTEM ARCHITECTURE

4.3 Deployment Architecture

The backend provides the application service layer. Java and Spring Boot handle requests, business operations, synchronization, and communication with the database. The backend becomes the central synchronization point when connectivity returns.

The database layer uses MySQL for persistent structured data. Synchronization requires careful handling of locally generated actions so that information collected during offline use can be transferred to the backend after reconnection.

The high-level architecture of AgriConnect can be viewed as four cooperating parts: the farmer or buyer interface, local offline storage, the Spring Boot backend, and the MySQL database. Dynamic market information and marketplace changes move between the client and backend when connectivity is available.

The frontend provides the user interaction layer. It displays crop advisory content and marketplace information and accepts actions such as product management and order management. Local storage keeps required data available when the network is unavailable.

The architecture is designed around continuity: the client can operate with local information, the backend coordinates online operations, and MySQL provides persistent structured data.

4.1 High-Level Architecture Diagram

The high-level architecture of AgriConnect can be viewed as four cooperating parts: the farmer or buyer interface, local offline storage, the Spring Boot backend, and the MySQL database. Dynamic market information and marketplace changes move between the client and backend when connectivity is available.

The frontend provides the user interaction layer. It displays crop advisory content and marketplace information and accepts actions such as product management and order management. Local storage keeps required data available when the network is unavailable.

The backend provides the application service layer. Java and Spring Boot handle requests, business operations, synchronization, and communication with the database. The backend becomes the central synchronization point when connectivity returns.

The database layer uses MySQL for persistent structured data. Synchronization requires careful handling of locally generated actions so that information collected during offline use can be transferred to the backend after reconnection.

The architecture is designed around continuity: the client can operate with local information, the backend coordinates online operations, and MySQL provides persistent structured data.

4.2 Description of Each Layer

The backend provides the application service layer. Java and Spring Boot handle requests, business operations, synchronization, and communication with the database. The backend becomes the central synchronization point when connectivity returns.

The database layer uses MySQL for persistent structured data. Synchronization requires careful handling of locally generated actions so that information collected during offline use can be transferred to the backend after reconnection.

The high-level architecture of AgriConnect can be viewed as four cooperating parts: the farmer or buyer interface, local offline storage, the Spring Boot backend, and the MySQL database. Dynamic market information and marketplace changes move between the client and backend when connectivity is available.

The frontend provides the user interaction layer. It displays crop advisory content and marketplace information and accepts actions such as product management and order management. Local storage keeps required data available when the network is unavailable.

The architecture is designed around continuity: the client can operate with local information, the backend coordinates online operations, and MySQL provides persistent structured data.

4.3 Deployment Architecture

The high-level architecture of AgriConnect can be viewed as four cooperating parts: the farmer or buyer interface, local offline storage, the Spring Boot backend, and the MySQL database. Dynamic market information and marketplace changes move between the client and backend when connectivity is available.

The frontend provides the user interaction layer. It displays crop advisory content and marketplace information and accepts actions such as product management and order management. Local storage keeps required data available when the network is unavailable.

The backend provides the application service layer. Java and Spring Boot handle requests, business operations, synchronization, and communication with the database. The backend becomes the central synchronization point when connectivity returns.

The database layer uses MySQL for persistent structured data. Synchronization requires careful handling of locally generated actions so that information collected during offline use can be transferred to the backend after reconnection.

The architecture is designed around continuity: the client can operate with local information, the backend coordinates online operations, and MySQL provides persistent structured data.

CHAPTER -5 DESIGN

5.1 System Design

The database design should maintain clear relationships between the farmer, product, buyer, and order records. Market information can be associated with the relevant crop and location so that updated information can be presented when synchronization occurs.

The design of AgriConnect is centered on the offline-first requirement. The system separates information that should remain available offline from information that must be refreshed when connectivity is restored.

The main entities can include farmers, products, buyers, orders, crop information, market information, and synchronization records. Farmer and product relationships support marketplace operations, while order records connect buyers and products.

The database design should support clear relationships between farmers, products, buyers, orders, crop information, market information, and synchronization records.

CHAPTER -6 IMPLEMENTATION

6.2 Front-End Logic

The frontend logic is implemented using HTML, CSS, and JavaScript. The backend API layer is implemented using Java and Spring Boot, while MySQL provides persistent storage. Postman can be used to verify API behavior during development.

The implementation is organized into modules that correspond to the functions described in the project review: crop advisory, farmer product management, buyer interaction and trust ratings, order management, market information, and offline synchronization.

The crop advisory module presents information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling. This information is intended to remain available offline so that a network interruption does not remove access to essential guidance.

The farmer product management module allows a farmer to manage products that are available for buyers. Product actions can be stored locally when offline and synchronized with the backend after connectivity is restored.

Module implementation should keep offline operations separate from synchronization logic while exposing the required marketplace and advisory functions through the backend API.

6.3 API Integration

The crop advisory module presents information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling. This information is intended to remain available offline so that a network interruption does not remove access to essential guidance.

The farmer product management module allows a farmer to manage products that are available for buyers. Product actions can be stored locally when offline and synchronized with the backend after connectivity is restored.

The buyer interaction and trust component allows farmers to view buyer information and trust ratings. The purpose is to give farmers additional information when interacting with buyers through the marketplace.

The order module supports order-related activities between farmers and buyers. Offline actions can be recorded locally and later synchronized with the backend, reducing the dependence on continuous network access.

Module implementation should keep offline operations separate from synchronization logic while exposing the required marketplace and advisory functions through the backend API.

6.4 Authentication & Authorization

The buyer interaction and trust component allows farmers to view buyer information and trust ratings. The purpose is to give farmers additional information when interacting with buyers through the marketplace.

The order module supports order-related activities between farmers and buyers. Offline actions can be recorded locally and later synchronized with the backend, reducing the dependence on continuous network access.

The market information module presents market prices and location-based market information when current data is available. Synchronization refreshes this dynamic information after a connection is established.

The frontend logic is implemented using HTML, CSS, and JavaScript. The backend API layer is implemented using Java and Spring Boot, while MySQL provides persistent storage. Postman can be used to verify API behavior during development.

Module implementation should keep offline operations separate from synchronization logic while exposing the required marketplace and advisory functions through the backend API.

6.1 Module-Wise Implementation

The market information module presents market prices and location-based market information when current data is available. Synchronization refreshes this dynamic information after a connection is established.

The frontend logic is implemented using HTML, CSS, and JavaScript. The backend API layer is implemented using Java and Spring Boot, while MySQL provides persistent storage. Postman can be used to verify API behavior during development.

The implementation is organized into modules that correspond to the functions described in the project review: crop advisory, farmer product management, buyer interaction and trust ratings, order management, market information, and offline synchronization.

The crop advisory module presents information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling. This information is intended to remain available offline so that a network interruption does not remove access to essential guidance.

Module implementation should keep offline operations separate from synchronization logic while exposing the required marketplace and advisory functions through the backend API.

6.2 Front-End Logic

The implementation is organized into modules that correspond to the functions described in the project review: crop advisory, farmer product management, buyer interaction and trust ratings, order management, market information, and offline synchronization.

The crop advisory module presents information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling. This information is intended to remain available offline so that a network interruption does not remove access to essential guidance.

The farmer product management module allows a farmer to manage products that are available for buyers. Product actions can be stored locally when offline and synchronized with the backend after connectivity is restored.

The buyer interaction and trust component allows farmers to view buyer information and trust ratings. The purpose is to give farmers additional information when interacting with buyers through the marketplace.

Module implementation should keep offline operations separate from synchronization logic while exposing the required marketplace and advisory functions through the backend API.

6.3 API Integration

The farmer product management module allows a farmer to manage products that are available for buyers. Product actions can be stored locally when offline and synchronized with the backend after connectivity is restored.

The buyer interaction and trust component allows farmers to view buyer information and trust ratings. The purpose is to give farmers additional information when interacting with buyers through the marketplace.

The order module supports order-related activities between farmers and buyers. Offline actions can be recorded locally and later synchronized with the backend, reducing the dependence on continuous network access.

The market information module presents market prices and location-based market information when current data is available. Synchronization refreshes this dynamic information after a connection is established.

Module implementation should keep offline operations separate from synchronization logic while exposing the required marketplace and advisory functions through the backend API.

6.4 Authentication & Authorization

The order module supports order-related activities between farmers and buyers. Offline actions can be recorded locally and later synchronized with the backend, reducing the dependence on continuous network access.

The market information module presents market prices and location-based market information when current data is available. Synchronization refreshes this dynamic information after a connection is established.

The frontend logic is implemented using HTML, CSS, and JavaScript. The backend API layer is implemented using Java and Spring Boot, while MySQL provides persistent storage. Postman can be used to verify API behavior during development.

The implementation is organized into modules that correspond to the functions described in the project review: crop advisory, farmer product management, buyer interaction and trust ratings, order management, market information, and offline synchronization.

Module implementation should keep offline operations separate from synchronization logic while exposing the required marketplace and advisory functions through the backend API.

6.1 Module-Wise Implementation

The frontend logic is implemented using HTML, CSS, and JavaScript. The backend API layer is implemented using Java and Spring Boot, while MySQL provides persistent storage. Postman can be used to verify API behavior during development.

The implementation is organized into modules that correspond to the functions described in the project review: crop advisory, farmer product management, buyer interaction and trust ratings, order management, market information, and offline synchronization.

The crop advisory module presents information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling. This information is intended to remain available offline so that a network interruption does not remove access to essential guidance.

The farmer product management module allows a farmer to manage products that are available for buyers. Product actions can be stored locally when offline and synchronized with the backend after connectivity is restored.

Module implementation should keep offline operations separate from synchronization logic while exposing the required marketplace and advisory functions through the backend API.

6.2 Front-End Logic

The crop advisory module presents information about planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling. This information is intended to remain available offline so that a network interruption does not remove access to essential guidance.

The farmer product management module allows a farmer to manage products that are available for buyers. Product actions can be stored locally when offline and synchronized with the backend after connectivity is restored.

The buyer interaction and trust component allows farmers to view buyer information and trust ratings. The purpose is to give farmers additional information when interacting with buyers through the marketplace.

The order module supports order-related activities between farmers and buyers. Offline actions can be recorded locally and later synchronized with the backend, reducing the dependence on continuous network access.

Module implementation should keep offline operations separate from synchronization logic while exposing the required marketplace and advisory functions through the backend API.

6.3 API Integration

The buyer interaction and trust component allows farmers to view buyer information and trust ratings. The purpose is to give farmers additional information when interacting with buyers through the marketplace.

The order module supports order-related activities between farmers and buyers. Offline actions can be recorded locally and later synchronized with the backend, reducing the dependence on continuous network access.

The market information module presents market prices and location-based market information when current data is available. Synchronization refreshes this dynamic information after a connection is established.

The frontend logic is implemented using HTML, CSS, and JavaScript. The backend API layer is implemented using Java and Spring Boot, while MySQL provides persistent storage. Postman can be used to verify API behavior during development.

Module implementation should keep offline operations separate from synchronization logic while exposing the required marketplace and advisory functions through the backend API.

CHAPTER 7- FEATURES

7.2 How Each Feature Works

Offline crop information is a central feature. Farmers can access guidance related to planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling even when a network connection is not available.

Product management allows farmers to maintain the products they want to offer through the marketplace. Buyer interaction adds information about buyers and their trust ratings, while order management supports marketplace transactions.

Market prices and location-based market information are updated whenever connectivity is available. The platform therefore combines persistent offline knowledge with dynamic information that is synchronized online.

Automatic synchronization is the connecting feature between offline and online use. Locally stored data and offline actions can be transferred to the backend when the network becomes available.

The features are intended to reduce dependence on continuous connectivity and to help farmers access information and marketplace functions in practical field conditions.

7.1 List of Main Features

Market prices and location-based market information are updated whenever connectivity is available. The platform therefore combines persistent offline knowledge with dynamic information that is synchronized online.

Automatic synchronization is the connecting feature between offline and online use. Locally stored data and offline actions can be transferred to the backend when the network becomes available.

AgriConnect combines agricultural advisory and marketplace functions in one offline-first platform. The main features are designed around the practical needs identified in the project review.

Offline crop information is a central feature. Farmers can access guidance related to planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling even when a network connection is not available.

The features are intended to reduce dependence on continuous connectivity and to help farmers access information and marketplace functions in practical field conditions.

7.2 How Each Feature Works

AgriConnect combines agricultural advisory and marketplace functions in one offline-first platform. The main features are designed around the practical needs identified in the project review.

Offline crop information is a central feature. Farmers can access guidance related to planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling even when a network connection is not available.

Product management allows farmers to maintain the products they want to offer through the marketplace. Buyer interaction adds information about buyers and their trust ratings, while order management supports marketplace transactions.

Market prices and location-based market information are updated whenever connectivity is available. The platform therefore combines persistent offline knowledge with dynamic information that is synchronized online.

The features are intended to reduce dependence on continuous connectivity and to help farmers access information and marketplace functions in practical field conditions.

7.1 List of Main Features

Product management allows farmers to maintain the products they want to offer through the marketplace. Buyer interaction adds information about buyers and their trust ratings, while order management supports marketplace transactions.

Market prices and location-based market information are updated whenever connectivity is available. The platform therefore combines persistent offline knowledge with dynamic information that is synchronized online.

Automatic synchronization is the connecting feature between offline and online use. Locally stored data and offline actions can be transferred to the backend when the network becomes available.

AgriConnect combines agricultural advisory and marketplace functions in one offline-first platform. The main features are designed around the practical needs identified in the project review.

The features are intended to reduce dependence on continuous connectivity and to help farmers access information and marketplace functions in practical field conditions.

7.2 How Each Feature Works

Automatic synchronization is the connecting feature between offline and online use. Locally stored data and offline actions can be transferred to the backend when the network becomes available.

AgriConnect combines agricultural advisory and marketplace functions in one offline-first platform. The main features are designed around the practical needs identified in the project review.

Offline crop information is a central feature. Farmers can access guidance related to planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling even when a network connection is not available.

Product management allows farmers to maintain the products they want to offer through the marketplace. Buyer interaction adds information about buyers and their trust ratings, while order management supports marketplace transactions.

The features are intended to reduce dependence on continuous connectivity and to help farmers access information and marketplace functions in practical field conditions.

7.1 List of Main Features

Offline crop information is a central feature. Farmers can access guidance related to planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling even when a network connection is not available.

Product management allows farmers to maintain the products they want to offer through the marketplace. Buyer interaction adds information about buyers and their trust ratings, while order management supports marketplace transactions.

Market prices and location-based market information are updated whenever connectivity is available. The platform therefore combines persistent offline knowledge with dynamic information that is synchronized online.

Automatic synchronization is the connecting feature between offline and online use. Locally stored data and offline actions can be transferred to the backend when the network becomes available.

The features are intended to reduce dependence on continuous connectivity and to help farmers access information and marketplace functions in practical field conditions.

7.2 How Each Feature Works

Market prices and location-based market information are updated whenever connectivity is available. The platform therefore combines persistent offline knowledge with dynamic information that is synchronized online.

Automatic synchronization is the connecting feature between offline and online use. Locally stored data and offline actions can be transferred to the backend when the network becomes available.

AgriConnect combines agricultural advisory and marketplace functions in one offline-first platform. The main features are designed around the practical needs identified in the project review.

Offline crop information is a central feature. Farmers can access guidance related to planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling even when a network connection is not available.

The features are intended to reduce dependence on continuous connectivity and to help farmers access information and marketplace functions in practical field conditions.

7.1 List of Main Features

AgriConnect combines agricultural advisory and marketplace functions in one offline-first platform. The main features are designed around the practical needs identified in the project review.

Offline crop information is a central feature. Farmers can access guidance related to planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling even when a network connection is not available.

Product management allows farmers to maintain the products they want to offer through the marketplace. Buyer interaction adds information about buyers and their trust ratings, while order management supports marketplace transactions.

Market prices and location-based market information are updated whenever connectivity is available. The platform therefore combines persistent offline knowledge with dynamic information that is synchronized online.

The features are intended to reduce dependence on continuous connectivity and to help farmers access information and marketplace functions in practical field conditions.

CHAPTER -8 TESTING

8.4 Test Cases and Results

Testing of AgriConnect should verify both ordinary online operation and the special offline-first behavior. Postman can be used to test the Spring Boot APIs, while integration testing can verify the interaction between frontend, backend, database, and synchronization logic.

API testing can cover farmer, product, buyer, order, market information, and synchronization operations. The objective is to check that requests are accepted correctly and that the expected responses are produced by the backend.

Offline testing should deliberately remove or interrupt connectivity and verify that crop information remains available and that supported actions are retained locally. Reconnection testing should then verify that local changes are synchronized with the backend.

Integration testing should confirm that MySQL persistence, Spring Boot services, frontend requests, and local storage work together. Test cases should include normal connectivity, low connectivity, no connectivity, reconnection, and repeated synchronization scenarios.

Testing should cover both connected and disconnected operation, followed by reconnection and synchronization checks. API behavior can be verified with Postman.

8.1 Postman Testing

Offline testing should deliberately remove or interrupt connectivity and verify that crop information remains available and that supported actions are retained locally.

Reconnection testing should then verify that local changes are synchronized with the backend.

Integration testing should confirm that MySQL persistence, Spring Boot services, frontend requests, and local storage work together. Test cases should include normal connectivity, low connectivity, no connectivity, reconnection, and repeated synchronization scenarios.

Testing of AgriConnect should verify both ordinary online operation and the special offline-first behavior. Postman can be used to test the Spring Boot APIs, while integration testing can verify the interaction between frontend, backend, database, and synchronization logic.

API testing can cover farmer, product, buyer, order, market information, and synchronization operations. The objective is to check that requests are accepted correctly and that the expected responses are produced by the backend.

Testing should cover both connected and disconnected operation, followed by reconnection and synchronization checks. API behavior can be verified with Postman.

8.2 Integration Testing

Testing of AgriConnect should verify both ordinary online operation and the special offline-first behavior. Postman can be used to test the Spring Boot APIs, while integration testing can verify the interaction between frontend, backend, database, and synchronization logic.

API testing can cover farmer, product, buyer, order, market information, and synchronization operations. The objective is to check that requests are accepted correctly and that the expected responses are produced by the backend.

Offline testing should deliberately remove or interrupt connectivity and verify that crop information remains available and that supported actions are retained locally. Reconnection testing should then verify that local changes are synchronized with the backend.

Integration testing should confirm that MySQL persistence, Spring Boot services, frontend requests, and local storage work together. Test cases should include normal connectivity, low connectivity, no connectivity, reconnection, and repeated synchronization scenarios.

Testing should cover both connected and disconnected operation, followed by reconnection and synchronization checks. API behavior can be verified with Postman.

8.3 Tools Used for Testing

Offline testing should deliberately remove or interrupt connectivity and verify that crop information remains available and that supported actions are retained locally.

Reconnection testing should then verify that local changes are synchronized with the backend.

Integration testing should confirm that MySQL persistence, Spring Boot services, frontend requests, and local storage work together. Test cases should include normal connectivity, low connectivity, no connectivity, reconnection, and repeated synchronization scenarios.

Testing of AgriConnect should verify both ordinary online operation and the special offline-first behavior. Postman can be used to test the Spring Boot APIs, while integration testing can verify the interaction between frontend, backend, database, and synchronization logic.

API testing can cover farmer, product, buyer, order, market information, and synchronization operations. The objective is to check that requests are accepted correctly and that the expected responses are produced by the backend.

Testing should cover both connected and disconnected operation, followed by reconnection and synchronization checks. API behavior can be verified with Postman.

8.4 Test Cases and Results

Testing of AgriConnect should verify both ordinary online operation and the special offline-first behavior. Postman can be used to test the Spring Boot APIs, while integration testing can verify the interaction between frontend, backend, database, and synchronization logic.

API testing can cover farmer, product, buyer, order, market information, and synchronization operations. The objective is to check that requests are accepted correctly and that the expected responses are produced by the backend.

Offline testing should deliberately remove or interrupt connectivity and verify that crop information remains available and that supported actions are retained locally. Reconnection testing should then verify that local changes are synchronized with the backend.

Integration testing should confirm that MySQL persistence, Spring Boot services, frontend requests, and local storage work together. Test cases should include normal connectivity, low connectivity, no connectivity, reconnection, and repeated synchronization scenarios.

Testing should cover both connected and disconnected operation, followed by reconnection and synchronization checks. API behavior can be verified with Postman.

8.1 Postman Testing

Offline testing should deliberately remove or interrupt connectivity and verify that crop information remains available and that supported actions are retained locally.

Reconnection testing should then verify that local changes are synchronized with the backend.

Integration testing should confirm that MySQL persistence, Spring Boot services, frontend requests, and local storage work together. Test cases should include normal connectivity, low connectivity, no connectivity, reconnection, and repeated synchronization scenarios.

Testing of AgriConnect should verify both ordinary online operation and the special offline-first behavior. Postman can be used to test the Spring Boot APIs, while integration testing can verify the interaction between frontend, backend, database, and synchronization logic.

API testing can cover farmer, product, buyer, order, market information, and synchronization operations. The objective is to check that requests are accepted correctly and that the expected responses are produced by the backend.

Testing should cover both connected and disconnected operation, followed by reconnection and synchronization checks. API behavior can be verified with Postman.

8.2 Integration Testing

Testing of AgriConnect should verify both ordinary online operation and the special offline-first behavior. Postman can be used to test the Spring Boot APIs, while integration testing can verify the interaction between frontend, backend, database, and synchronization logic.

API testing can cover farmer, product, buyer, order, market information, and synchronization operations. The objective is to check that requests are accepted correctly and that the expected responses are produced by the backend.

Offline testing should deliberately remove or interrupt connectivity and verify that crop information remains available and that supported actions are retained locally. Reconnection testing should then verify that local changes are synchronized with the backend.

Integration testing should confirm that MySQL persistence, Spring Boot services, frontend requests, and local storage work together. Test cases should include normal connectivity, low connectivity, no connectivity, reconnection, and repeated synchronization scenarios.

Testing should cover both connected and disconnected operation, followed by reconnection and synchronization checks. API behavior can be verified with Postman.

8.3 Tools Used for Testing

Offline testing should deliberately remove or interrupt connectivity and verify that crop information remains available and that supported actions are retained locally.

Reconnection testing should then verify that local changes are synchronized with the backend.

Integration testing should confirm that MySQL persistence, Spring Boot services, frontend requests, and local storage work together. Test cases should include normal connectivity, low connectivity, no connectivity, reconnection, and repeated synchronization scenarios.

Testing of AgriConnect should verify both ordinary online operation and the special offline-first behavior. Postman can be used to test the Spring Boot APIs, while integration testing can verify the interaction between frontend, backend, database, and synchronization logic.

API testing can cover farmer, product, buyer, order, market information, and synchronization operations. The objective is to check that requests are accepted correctly and that the expected responses are produced by the backend.

Testing should cover both connected and disconnected operation, followed by reconnection and synchronization checks. API behavior can be verified with Postman.

8.4 Test Cases and Results

Testing of AgriConnect should verify both ordinary online operation and the special offline-first behavior. Postman can be used to test the Spring Boot APIs, while integration testing can verify the interaction between frontend, backend, database, and synchronization logic.

API testing can cover farmer, product, buyer, order, market information, and synchronization operations. The objective is to check that requests are accepted correctly and that the expected responses are produced by the backend.

Offline testing should deliberately remove or interrupt connectivity and verify that crop information remains available and that supported actions are retained locally. Reconnection testing should then verify that local changes are synchronized with the backend.

Integration testing should confirm that MySQL persistence, Spring Boot services, frontend requests, and local storage work together. Test cases should include normal connectivity, low connectivity, no connectivity, reconnection, and repeated synchronization scenarios.

Testing should cover both connected and disconnected operation, followed by reconnection and synchronization checks. API behavior can be verified with Postman.

CHAPTER -9 DEPLOYMENT

9.2 Environment Configuration

The frontend can be served from a compatible web server or local development environment. Its local storage capability remains important because the offline-first behavior must continue after deployment.

After deployment, the system should be checked for API availability, database connectivity, frontend access, offline operation, and synchronization after reconnection. Postman can be used to verify the deployed backend APIs.

Deployment of AgriConnect involves preparing the frontend, Spring Boot backend, MySQL database, and offline storage behavior. The exact hosting environment is not specified in the project review, so deployment should be treated as an implementation consideration rather than a claimed completed deployment.

The backend is built using Maven and Java with Spring Boot. The database connection is configured for MySQL. Environment-specific settings such as database connection details should be kept separate from application source code where appropriate.

The exact production hosting arrangement is not specified in the review material; deployment should therefore be documented as the selected implementation environment is established.

9.3 Hosting of Frontend and Backend

Deployment of AgriConnect involves preparing the frontend, Spring Boot backend, MySQL database, and offline storage behavior. The exact hosting environment is not specified in the project review, so deployment should be treated as an implementation consideration rather than a claimed completed deployment.

The backend is built using Maven and Java with Spring Boot. The database connection is configured for MySQL. Environment-specific settings such as database connection details should be kept separate from application source code where appropriate.

The frontend can be served from a compatible web server or local development environment. Its local storage capability remains important because the offline-first behavior must continue after deployment.

After deployment, the system should be checked for API availability, database connectivity, frontend access, offline operation, and synchronization after reconnection. Postman can be used to verify the deployed backend APIs.

The exact production hosting arrangement is not specified in the review material; deployment should therefore be documented as the selected implementation environment is established.

9.1 Steps to Deploy

The frontend can be served from a compatible web server or local development environment. Its local storage capability remains important because the offline-first behavior must continue after deployment.

After deployment, the system should be checked for API availability, database connectivity, frontend access, offline operation, and synchronization after reconnection. Postman can be used to verify the deployed backend APIs.

Deployment of AgriConnect involves preparing the frontend, Spring Boot backend, MySQL database, and offline storage behavior. The exact hosting environment is not specified in the project review, so deployment should be treated as an implementation consideration rather than a claimed completed deployment.

The backend is built using Maven and Java with Spring Boot. The database connection is configured for MySQL. Environment-specific settings such as database connection details should be kept separate from application source code where appropriate.

The exact production hosting arrangement is not specified in the review material; deployment should therefore be documented as the selected implementation environment is established.

9.2 Environment Configuration

Deployment of AgriConnect involves preparing the frontend, Spring Boot backend, MySQL database, and offline storage behavior. The exact hosting environment is not specified in the project review, so deployment should be treated as an implementation consideration rather than a claimed completed deployment.

The backend is built using Maven and Java with Spring Boot. The database connection is configured for MySQL. Environment-specific settings such as database connection details should be kept separate from application source code where appropriate.

The frontend can be served from a compatible web server or local development environment. Its local storage capability remains important because the offline-first behavior must continue after deployment.

After deployment, the system should be checked for API availability, database connectivity, frontend access, offline operation, and synchronization after reconnection. Postman can be used to verify the deployed backend APIs.

The exact production hosting arrangement is not specified in the review material; deployment should therefore be documented as the selected implementation environment is established.

9.3 Hosting of Frontend and Backend

The frontend can be served from a compatible web server or local development environment. Its local storage capability remains important because the offline-first behavior must continue after deployment.

After deployment, the system should be checked for API availability, database connectivity, frontend access, offline operation, and synchronization after reconnection. Postman can be used to verify the deployed backend APIs.

Deployment of AgriConnect involves preparing the frontend, Spring Boot backend, MySQL database, and offline storage behavior. The exact hosting environment is not specified in the project review, so deployment should be treated as an implementation consideration rather than a claimed completed deployment.

The backend is built using Maven and Java with Spring Boot. The database connection is configured for MySQL. Environment-specific settings such as database connection details should be kept separate from application source code where appropriate.

The exact production hosting arrangement is not specified in the review material; deployment should therefore be documented as the selected implementation environment is established.

9.1 Steps to Deploy

Deployment of AgriConnect involves preparing the frontend, Spring Boot backend, MySQL database, and offline storage behavior. The exact hosting environment is not specified in the project review, so deployment should be treated as an implementation consideration rather than a claimed completed deployment.

The backend is built using Maven and Java with Spring Boot. The database connection is configured for MySQL. Environment-specific settings such as database connection details should be kept separate from application source code where appropriate.

The frontend can be served from a compatible web server or local development environment. Its local storage capability remains important because the offline-first behavior must continue after deployment.

After deployment, the system should be checked for API availability, database connectivity, frontend access, offline operation, and synchronization after reconnection. Postman can be used to verify the deployed backend APIs.

The exact production hosting arrangement is not specified in the review material; deployment should therefore be documented as the selected implementation environment is established.

9.2 Environment Configuration

The frontend can be served from a compatible web server or local development environment. Its local storage capability remains important because the offline-first behavior must continue after deployment.

After deployment, the system should be checked for API availability, database connectivity, frontend access, offline operation, and synchronization after reconnection. Postman can be used to verify the deployed backend APIs.

Deployment of AgriConnect involves preparing the frontend, Spring Boot backend, MySQL database, and offline storage behavior. The exact hosting environment is not specified in the project review, so deployment should be treated as an implementation consideration rather than a claimed completed deployment.

The backend is built using Maven and Java with Spring Boot. The database connection is configured for MySQL. Environment-specific settings such as database connection details should be kept separate from application source code where appropriate.

The exact production hosting arrangement is not specified in the review material; deployment should therefore be documented as the selected implementation environment is established.

9.3 Hosting of Frontend and Backend

Deployment of AgriConnect involves preparing the frontend, Spring Boot backend, MySQL database, and offline storage behavior. The exact hosting environment is not specified in the project review, so deployment should be treated as an implementation consideration rather than a claimed completed deployment.

The backend is built using Maven and Java with Spring Boot. The database connection is configured for MySQL. Environment-specific settings such as database connection details should be kept separate from application source code where appropriate.

The frontend can be served from a compatible web server or local development environment. Its local storage capability remains important because the offline-first behavior must continue after deployment.

After deployment, the system should be checked for API availability, database connectivity, frontend access, offline operation, and synchronization after reconnection. Postman can be used to verify the deployed backend APIs.

The exact production hosting arrangement is not specified in the review material; deployment should therefore be documented as the selected implementation environment is established.

9.1 Steps to Deploy

The frontend can be served from a compatible web server or local development environment. Its local storage capability remains important because the offline-first behavior must continue after deployment.

After deployment, the system should be checked for API availability, database connectivity, frontend access, offline operation, and synchronization after reconnection. Postman can be used to verify the deployed backend APIs.

Deployment of AgriConnect involves preparing the frontend, Spring Boot backend, MySQL database, and offline storage behavior. The exact hosting environment is not specified in the project review, so deployment should be treated as an implementation consideration rather than a claimed completed deployment.

The backend is built using Maven and Java with Spring Boot. The database connection is configured for MySQL. Environment-specific settings such as database connection details should be kept separate from application source code where appropriate.

The exact production hosting arrangement is not specified in the review material; deployment should therefore be documented as the selected implementation environment is established.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.2 Solutions Applied

Usability is also important because the platform is intended for farmers in rural and remote areas. The interface should keep the main functions clear and reduce unnecessary dependence on network availability.

The current project review does not specify completed production-scale testing, hosting, or measured performance results. These areas remain implementation and validation considerations for later stages.

The main development challenge is unreliable connectivity. An application that assumes continuous internet access cannot provide the required experience in low-network areas. AgriConnect addresses this by storing essential information locally and synchronizing dynamic information later.

Another challenge is maintaining consistency between local and server-side information. Offline actions can exist on the device before they reach the backend, so synchronization must transfer those actions in an organized way.

The project review identifies connectivity as the central challenge. Local storage and synchronization are the main design responses, while production-scale limitations remain to be validated.

10.3 Current Limitations

The main development challenge is unreliable connectivity. An application that assumes continuous internet access cannot provide the required experience in low-network areas. AgriConnect addresses this by storing essential information locally and synchronizing dynamic information later.

Another challenge is maintaining consistency between local and server-side information. Offline actions can exist on the device before they reach the backend, so synchronization must transfer those actions in an organized way.

Market prices are dynamic and therefore cannot be treated exactly like static crop guidance. The system must distinguish reusable offline information from data that should be refreshed whenever a connection is available.

Usability is also important because the platform is intended for farmers in rural and remote areas. The interface should keep the main functions clear and reduce unnecessary dependence on network availability.

The project review identifies connectivity as the central challenge. Local storage and synchronization are the main design responses, while production-scale limitations remain to be validated.

10.1 Issues Faced During Development

Market prices are dynamic and therefore cannot be treated exactly like static crop guidance. The system must distinguish reusable offline information from data that should be refreshed whenever a connection is available.

Usability is also important because the platform is intended for farmers in rural and remote areas. The interface should keep the main functions clear and reduce unnecessary dependence on network availability.

The current project review does not specify completed production-scale testing, hosting, or measured performance results. These areas remain implementation and validation considerations for later stages.

The main development challenge is unreliable connectivity. An application that assumes continuous internet access cannot provide the required experience in low-network areas. AgriConnect addresses this by storing essential information locally and synchronizing dynamic information later.

The project review identifies connectivity as the central challenge. Local storage and synchronization are the main design responses, while production-scale limitations remain to be validated.

10.2 Solutions Applied

The current project review does not specify completed production-scale testing, hosting, or measured performance results. These areas remain implementation and validation considerations for later stages.

The main development challenge is unreliable connectivity. An application that assumes continuous internet access cannot provide the required experience in low-network areas. AgriConnect addresses this by storing essential information locally and synchronizing dynamic information later.

Another challenge is maintaining consistency between local and server-side information. Offline actions can exist on the device before they reach the backend, so synchronization must transfer those actions in an organized way.

Market prices are dynamic and therefore cannot be treated exactly like static crop guidance. The system must distinguish reusable offline information from data that should be refreshed whenever a connection is available.

The project review identifies connectivity as the central challenge. Local storage and synchronization are the main design responses, while production-scale limitations remain to be validated.

10.3 Current Limitations

Another challenge is maintaining consistency between local and server-side information. Offline actions can exist on the device before they reach the backend, so synchronization must transfer those actions in an organized way.

Market prices are dynamic and therefore cannot be treated exactly like static crop guidance. The system must distinguish reusable offline information from data that should be refreshed whenever a connection is available.

Usability is also important because the platform is intended for farmers in rural and remote areas. The interface should keep the main functions clear and reduce unnecessary dependence on network availability.

The current project review does not specify completed production-scale testing, hosting, or measured performance results. These areas remain implementation and validation considerations for later stages.

The project review identifies connectivity as the central challenge. Local storage and synchronization are the main design responses, while production-scale limitations remain to be validated.

10.1 Issues Faced During Development

Usability is also important because the platform is intended for farmers in rural and remote areas. The interface should keep the main functions clear and reduce unnecessary dependence on network availability.

The current project review does not specify completed production-scale testing, hosting, or measured performance results. These areas remain implementation and validation considerations for later stages.

The main development challenge is unreliable connectivity. An application that assumes continuous internet access cannot provide the required experience in low-network areas. AgriConnect addresses this by storing essential information locally and synchronizing dynamic information later.

Another challenge is maintaining consistency between local and server-side information. Offline actions can exist on the device before they reach the backend, so synchronization must transfer those actions in an organized way.

The project review identifies connectivity as the central challenge. Local storage and synchronization are the main design responses, while production-scale limitations remain to be validated.

10.2 Solutions Applied

The main development challenge is unreliable connectivity. An application that assumes continuous internet access cannot provide the required experience in low-network areas. AgriConnect addresses this by storing essential information locally and synchronizing dynamic information later.

Another challenge is maintaining consistency between local and server-side information. Offline actions can exist on the device before they reach the backend, so synchronization must transfer those actions in an organized way.

Market prices are dynamic and therefore cannot be treated exactly like static crop guidance. The system must distinguish reusable offline information from data that should be refreshed whenever a connection is available.

Usability is also important because the platform is intended for farmers in rural and remote areas. The interface should keep the main functions clear and reduce unnecessary dependence on network availability.

The project review identifies connectivity as the central challenge. Local storage and synchronization are the main design responses, while production-scale limitations remain to be validated.

10.3 Current Limitations

Market prices are dynamic and therefore cannot be treated exactly like static crop guidance. The system must distinguish reusable offline information from data that should be refreshed whenever a connection is available.

Usability is also important because the platform is intended for farmers in rural and remote areas. The interface should keep the main functions clear and reduce unnecessary dependence on network availability.

The current project review does not specify completed production-scale testing, hosting, or measured performance results. These areas remain implementation and validation considerations for later stages.

The main development challenge is unreliable connectivity. An application that assumes continuous internet access cannot provide the required experience in low-network areas. AgriConnect addresses this by storing essential information locally and synchronizing dynamic information later.

The project review identifies connectivity as the central challenge. Local storage and synchronization are the main design responses, while production-scale limitations remain to be validated.

10.1 Issues Faced During Development

The current project review does not specify completed production-scale testing, hosting, or measured performance results. These areas remain implementation and validation considerations for later stages.

The main development challenge is unreliable connectivity. An application that assumes continuous internet access cannot provide the required experience in low-network areas. AgriConnect addresses this by storing essential information locally and synchronizing dynamic information later.

Another challenge is maintaining consistency between local and server-side information. Offline actions can exist on the device before they reach the backend, so synchronization must transfer those actions in an organized way.

Market prices are dynamic and therefore cannot be treated exactly like static crop guidance. The system must distinguish reusable offline information from data that should be refreshed whenever a connection is available.

The project review identifies connectivity as the central challenge. Local storage and synchronization are the main design responses, while production-scale limitations remain to be validated.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

Weather and market-price prediction are potential future capabilities. These could provide additional decision support beyond the current synchronization of market information.

Regional language and voice support can improve accessibility for farmers who prefer local-language interaction. Such support could be added to the user interface and advisory content.

IoT-based crop monitoring is another future direction. Sensor information could be integrated with the advisory system to provide more timely information about crop conditions.

Integration with government agricultural services could connect farmers to additional agricultural information and services, extending the usefulness of the platform beyond the current marketplace and advisory functions.

Future scope explicitly includes AI-based disease detection, weather and market-price prediction, regional language and voice support, IoT crop monitoring, and government agricultural services.

11.2 Possible Integrations or Optimizations

IoT-based crop monitoring is another future direction. Sensor information could be integrated with the advisory system to provide more timely information about crop conditions.

Integration with government agricultural services could connect farmers to additional agricultural information and services, extending the usefulness of the platform beyond the current marketplace and advisory functions.

The project identifies several future enhancements. AI-based disease detection can extend the advisory capability by helping identify crop diseases from images or other inputs, subject to suitable data and model development.

Weather and market-price prediction are potential future capabilities. These could provide additional decision support beyond the current synchronization of market information.

Future scope explicitly includes AI-based disease detection, weather and market-price prediction, regional language and voice support, IoT crop monitoring, and government agricultural services.

11.1 Planned Features

The project identifies several future enhancements. AI-based disease detection can extend the advisory capability by helping identify crop diseases from images or other inputs, subject to suitable data and model development.

Weather and market-price prediction are potential future capabilities. These could provide additional decision support beyond the current synchronization of market information.

Regional language and voice support can improve accessibility for farmers who prefer local-language interaction. Such support could be added to the user interface and advisory content.

IoT-based crop monitoring is another future direction. Sensor information could be integrated with the advisory system to provide more timely information about crop conditions.

Future scope explicitly includes AI-based disease detection, weather and market-price prediction, regional language and voice support, IoT crop monitoring, and government agricultural services.

11.2 Possible Integrations or Optimizations

Regional language and voice support can improve accessibility for farmers who prefer local-language interaction. Such support could be added to the user interface and advisory content.

IoT-based crop monitoring is another future direction. Sensor information could be integrated with the advisory system to provide more timely information about crop conditions.

Integration with government agricultural services could connect farmers to additional agricultural information and services, extending the usefulness of the platform beyond the current marketplace and advisory functions.

The project identifies several future enhancements. AI-based disease detection can extend the advisory capability by helping identify crop diseases from images or other inputs, subject to suitable data and model development.

Future scope explicitly includes AI-based disease detection, weather and market-price prediction, regional language and voice support, IoT crop monitoring, and government agricultural services.

11.1 Planned Features

Integration with government agricultural services could connect farmers to additional agricultural information and services, extending the usefulness of the platform beyond the current marketplace and advisory functions.

The project identifies several future enhancements. AI-based disease detection can extend the advisory capability by helping identify crop diseases from images or other inputs, subject to suitable data and model development.

Weather and market-price prediction are potential future capabilities. These could provide additional decision support beyond the current synchronization of market information.

Regional language and voice support can improve accessibility for farmers who prefer local-language interaction. Such support could be added to the user interface and advisory content.

Future scope explicitly includes AI-based disease detection, weather and market-price prediction, regional language and voice support, IoT crop monitoring, and government agricultural services.

11.2 Possible Integrations or Optimizations

Weather and market-price prediction are potential future capabilities. These could provide additional decision support beyond the current synchronization of market information.

Regional language and voice support can improve accessibility for farmers who prefer local-language interaction. Such support could be added to the user interface and advisory content.

IoT-based crop monitoring is another future direction. Sensor information could be integrated with the advisory system to provide more timely information about crop conditions.

Integration with government agricultural services could connect farmers to additional agricultural information and services, extending the usefulness of the platform beyond the current marketplace and advisory functions.

Future scope explicitly includes AI-based disease detection, weather and market-price prediction, regional language and voice support, IoT crop monitoring, and government agricultural services.

CHAPTER -12 CONCLUSION

12.2 What Was Achieved

The proposed technology stack consists of HTML, CSS, and JavaScript for the frontend; Java and Spring Boot for the backend; MySQL for the database; Postman for API testing; Git and GitHub for version control; Maven as the build tool; and Local Storage or Indexed DB for offline storage.

The project demonstrates how an offline-first design can reduce dependence on continuous connectivity while still using an online backend for synchronization and dynamic market information. The future scope further extends the system toward AI, prediction, voice support, IoT monitoring, and government-service integration.

AgriConnect provides an offline-first platform that combines agricultural advisory information, marketplace functions, and synchronized market information. The project is designed for farmers who may not have reliable continuous internet connectivity.

The system keeps essential crop knowledge available offline, covering planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. This directly addresses the information-access problem identified in the project review.

Overall, the project combines advisory information, marketplace functions, and synchronization in a single offline-first platform for farmers in low-connectivity areas.

12.3 Skills Learned During Development

AgriConnect provides an offline-first platform that combines agricultural advisory information, marketplace functions, and synchronized market information. The project is designed for farmers who may not have reliable continuous internet connectivity.

The system keeps essential crop knowledge available offline, covering planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. This directly addresses the information-access problem identified in the project review.

Marketplace functions allow farmers to manage products, interact with buyers, view trust ratings, and manage orders. Market prices and other dynamic information can be refreshed whenever connectivity becomes available.

The proposed technology stack consists of HTML, CSS, and JavaScript for the frontend; Java and Spring Boot for the backend; MySQL for the database; Postman for API testing; Git and GitHub for version control; Maven as the build tool; and Local Storage or Indexed DB for offline storage.

Overall, the project combines advisory information, marketplace functions, and synchronization in a single offline-first platform for farmers in low-connectivity areas.

12.1 Summary of the Project

Marketplace functions allow farmers to manage products, interact with buyers, view trust ratings, and manage orders. Market prices and other dynamic information can be refreshed whenever connectivity becomes available.

The proposed technology stack consists of HTML, CSS, and JavaScript for the frontend; Java and Spring Boot for the backend; MySQL for the database; Postman for API testing; Git and GitHub for version control; Maven as the build tool; and Local Storage or Indexed DB for offline storage.

The project demonstrates how an offline-first design can reduce dependence on continuous connectivity while still using an online backend for synchronization and dynamic market information. The future scope further extends the system toward AI, prediction, voice support, IoT monitoring, and government-service integration.

AgriConnect provides an offline-first platform that combines agricultural advisory information, marketplace functions, and synchronized market information. The project is designed for farmers who may not have reliable continuous internet connectivity.

Overall, the project combines advisory information, marketplace functions, and synchronization in a single offline-first platform for farmers in low-connectivity areas.

12.2 What Was Achieved

The project demonstrates how an offline-first design can reduce dependence on continuous connectivity while still using an online backend for synchronization and dynamic market information. The future scope further extends the system toward AI, prediction, voice support, IoT monitoring, and government-service integration.

AgriConnect provides an offline-first platform that combines agricultural advisory information, marketplace functions, and synchronized market information. The project is designed for farmers who may not have reliable continuous internet connectivity.

The system keeps essential crop knowledge available offline, covering planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. This directly addresses the information-access problem identified in the project review.

Marketplace functions allow farmers to manage products, interact with buyers, view trust ratings, and manage orders. Market prices and other dynamic information can be refreshed whenever connectivity becomes available.

Overall, the project combines advisory information, marketplace functions, and synchronization in a single offline-first platform for farmers in low-connectivity areas.

12.3 Skills Learned During Development

The system keeps essential crop knowledge available offline, covering planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. This directly addresses the information-access problem identified in the project review.

Marketplace functions allow farmers to manage products, interact with buyers, view trust ratings, and manage orders. Market prices and other dynamic information can be refreshed whenever connectivity becomes available.

The proposed technology stack consists of HTML, CSS, and JavaScript for the frontend; Java and Spring Boot for the backend; MySQL for the database; Postman for API testing; Git and GitHub for version control; Maven as the build tool; and Local Storage or Indexed DB for offline storage.

The project demonstrates how an offline-first design can reduce dependence on continuous connectivity while still using an online backend for synchronization and dynamic market information. The future scope further extends the system toward AI, prediction, voice support, IoT monitoring, and government-service integration.

Overall, the project combines advisory information, marketplace functions, and synchronization in a single offline-first platform for farmers in low-connectivity areas.

12.1 Summary of the Project

The proposed technology stack consists of HTML, CSS, and JavaScript for the frontend; Java and Spring Boot for the backend; MySQL for the database; Postman for API testing; Git and GitHub for version control; Maven as the build tool; and Local Storage or Indexed DB for offline storage.

The project demonstrates how an offline-first design can reduce dependence on continuous connectivity while still using an online backend for synchronization and dynamic market information. The future scope further extends the system toward AI, prediction, voice support, IoT monitoring, and government-service integration.

AgriConnect provides an offline-first platform that combines agricultural advisory information, marketplace functions, and synchronized market information. The project is designed for farmers who may not have reliable continuous internet connectivity.

The system keeps essential crop knowledge available offline, covering planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. This directly addresses the information-access problem identified in the project review.

Overall, the project combines advisory information, marketplace functions, and synchronization in a single offline-first platform for farmers in low-connectivity areas.

12.2 What Was Achieved

AgriConnect provides an offline-first platform that combines agricultural advisory information, marketplace functions, and synchronized market information. The project is designed for farmers who may not have reliable continuous internet connectivity.

The system keeps essential crop knowledge available offline, covering planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. This directly addresses the information-access problem identified in the project review.

Marketplace functions allow farmers to manage products, interact with buyers, view trust ratings, and manage orders. Market prices and other dynamic information can be refreshed whenever connectivity becomes available.

The proposed technology stack consists of HTML, CSS, and JavaScript for the frontend; Java and Spring Boot for the backend; MySQL for the database; Postman for API testing; Git and GitHub for version control; Maven as the build tool; and Local Storage or Indexed DB for offline storage.

Overall, the project combines advisory information, marketplace functions, and synchronization in a single offline-first platform for farmers in low-connectivity areas.

12.3 Skills Learned During Development

Marketplace functions allow farmers to manage products, interact with buyers, view trust ratings, and manage orders. Market prices and other dynamic information can be refreshed whenever connectivity becomes available.

The proposed technology stack consists of HTML, CSS, and JavaScript for the frontend; Java and Spring Boot for the backend; MySQL for the database; Postman for API testing; Git and GitHub for version control; Maven as the build tool; and Local Storage or Indexed DB for offline storage.

The project demonstrates how an offline-first design can reduce dependence on continuous connectivity while still using an online backend for synchronization and dynamic market information. The future scope further extends the system toward AI, prediction, voice support, IoT monitoring, and government-service integration.

AgriConnect provides an offline-first platform that combines agricultural advisory information, marketplace functions, and synchronized market information. The project is designed for farmers who may not have reliable continuous internet connectivity.

Overall, the project combines advisory information, marketplace functions, and synchronization in a single offline-first platform for farmers in low-connectivity areas.

12.1 Summary of the Project

The project demonstrates how an offline-first design can reduce dependence on continuous connectivity while still using an online backend for synchronization and dynamic market information. The future scope further extends the system toward AI, prediction, voice support, IoT monitoring, and government-service integration.

AgriConnect provides an offline-first platform that combines agricultural advisory information, marketplace functions, and synchronized market information. The project is designed for farmers who may not have reliable continuous internet connectivity.

The system keeps essential crop knowledge available offline, covering planting, fertilizers, irrigation, diseases, pests, harvesting, storage, and selling guidance. This directly addresses the information-access problem identified in the project review.

Marketplace functions allow farmers to manage products, interact with buyers, view trust ratings, and manage orders. Market prices and other dynamic information can be refreshed whenever connectivity becomes available.

Overall, the project combines advisory information, marketplace functions, and synchronization in a single offline-first platform for farmers in low-connectivity areas.

CHAPTER -13 REFERENCES

Documentation and Learning Resources

The project documentation is based primarily on the AgriConnect project review presentation. The presentation defines the project title, abstract, introduction, existing system, problem, proposed solution, objectives, modules, software requirements, and future scope.

Technology references for implementation include the official documentation and learning resources for Java, Spring Boot, MySQL, HTML, CSS, JavaScript, Postman, Git, GitHub, Maven, Local Storage, and Indexed DB. These resources can be consulted during implementation and testing.

These references should be replaced or supplemented with the exact documentation actually used during implementation and testing.

Project References

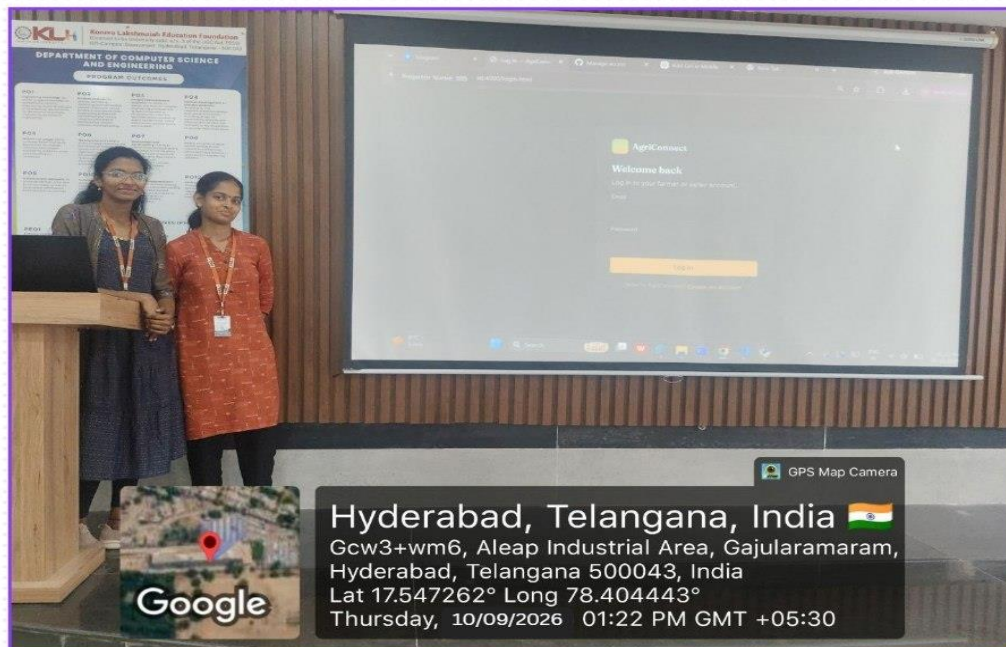
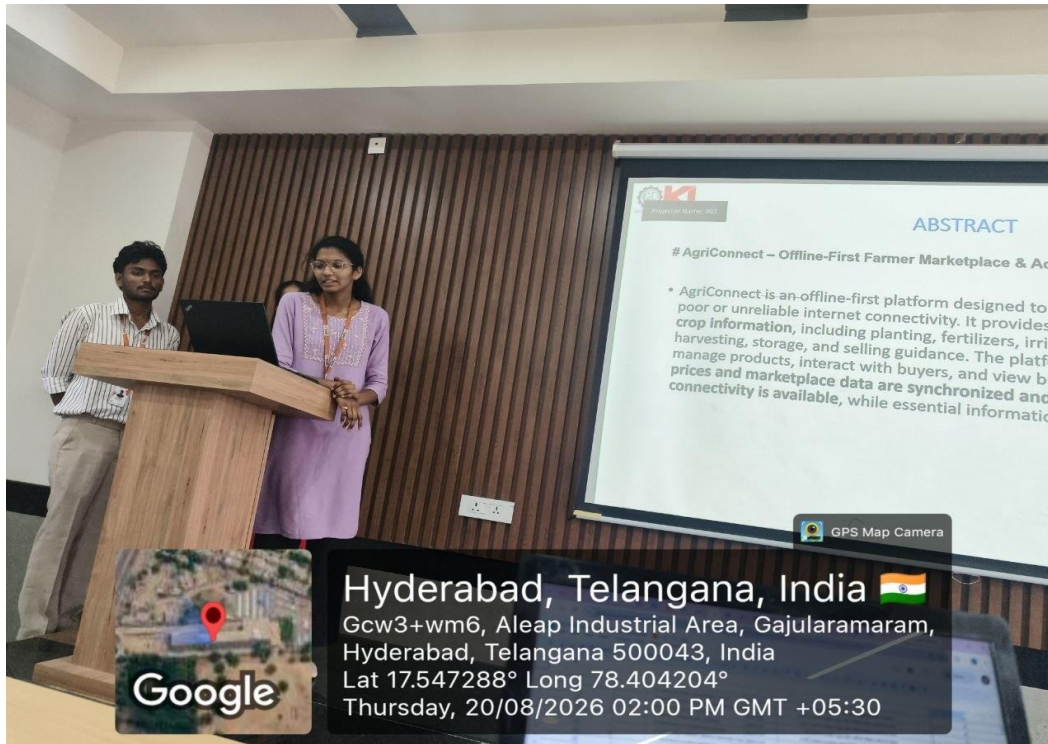
The project documentation is based primarily on the AgriConnect project review presentation. The presentation defines the project title, abstract, introduction, existing system, problem, proposed solution, objectives, modules, software requirements, and future scope.

Technology references for implementation include the official documentation and learning resources for Java, Spring Boot, MySQL, HTML, CSS, JavaScript, Postman, Git, GitHub, Maven, Local Storage, and Indexed DB. These resources can be consulted during implementation and testing.

These references should be replaced or supplemented with the exact documentation actually used during implementation and testing.

CHAPTER -14 APPENDICES

Geo-Tag Photos:



Review Forms with Guide Signatures

The appendices preserve the documentation categories used in the provided project-report format. They can contain screenshots of the AgriConnect interface, sample code snippets, installation and setup instructions, a user guide, geo-tag photos with guide details, and review forms with guide signatures.

Where screenshots or signed forms are not yet available, the appendix page acts as a placeholder in the same report structure. Actual project screenshots and signed documents can be inserted into these pages without changing the report formatting.

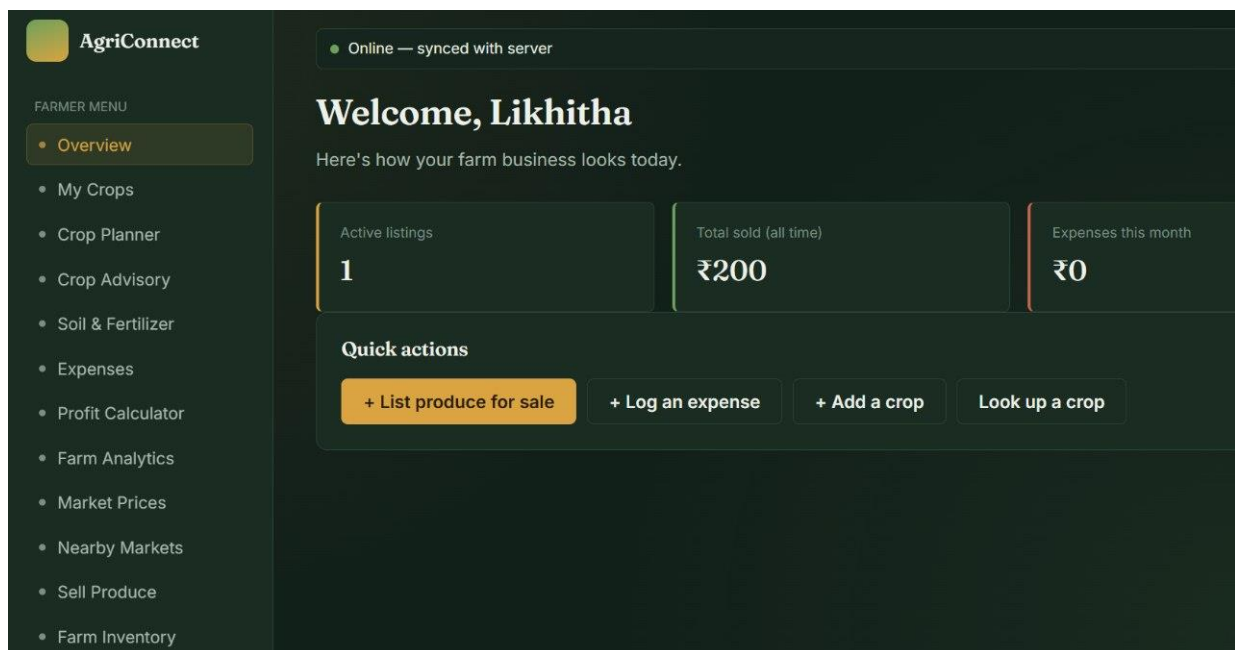
Actual screenshots, code listings, installation evidence, geo-tag photographs, and signed review forms can be inserted into the appendix without changing the template structure.

Screenshots of the App

The appendices preserve the documentation categories used in the provided project-report format. They can contain screenshots of the AgriConnect interface, sample code snippets, installation and setup instructions, a user guide, geo-tag photos with guide details, and review forms with guide signatures.

Where screenshots or signed forms are not yet available, the appendix page acts as a placeholder in the same report structure. Actual project screenshots and signed documents can be inserted into these pages without changing the report formatting.

Actual screenshots, code listings, installation evidence, geo-tag photographs, and signed review forms can be inserted into the appendix without changing the template structure.

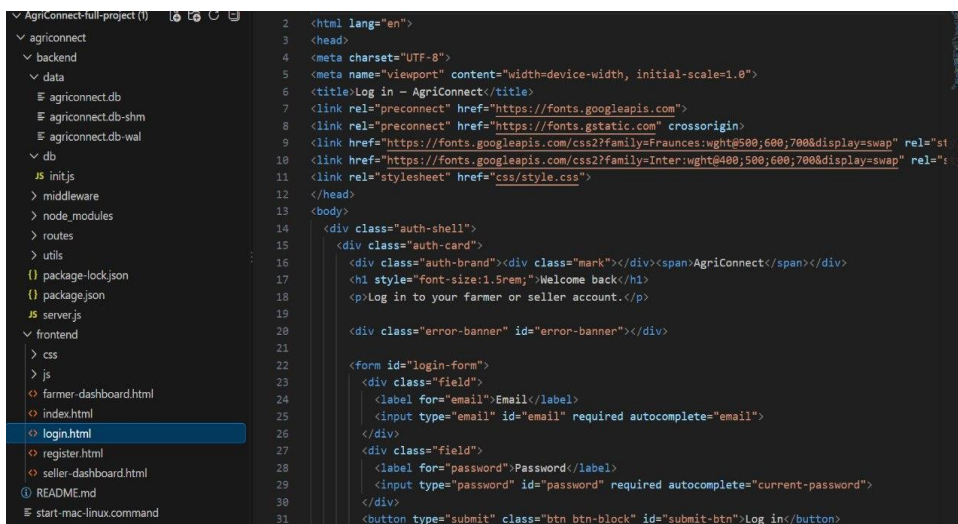


Sample Code Snippets

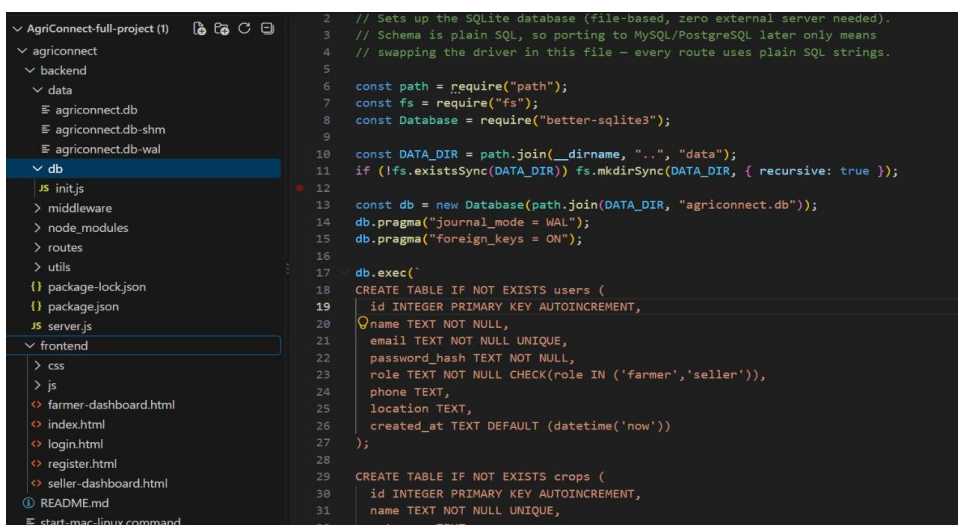
The appendices preserve the documentation categories used in the provided project-report format. They can contain screenshots of the AgriConnect interface, sample code snippets, installation and setup instructions, a user guide, geo-tag photos with guide details, and review forms with guide signatures.

Where screenshots or signed forms are not yet available, the appendix page acts as a placeholder in the same report structure. Actual project screenshots and signed documents can be inserted into these pages without changing the report formatting.

Actual screenshots, code listings, installation evidence, geo-tag photographs, and signed review forms can be inserted into the appendix without changing the template structure.



```
1 <html lang="en">
2 <head>
3 <meta charset="UTF-8">
4 <meta name="viewport" content="width=device-width, initial-scale=1.0">
5 <title>Log in - AgriConnect</title>
6 <link rel="preconnect" href="https://fonts.googleapis.com">
7 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
8 <link href="https://fonts.googleapis.com/css2?family=Fraunces:wght@500;600;700&display=swap" rel="stylesheet">
9 <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap" rel="stylesheet">
10 <link rel="stylesheet" href="css/style.css">
11 </head>
12 <body>
13 <div class="auth-shell">
14 <div class="auth-card">
15 <div class="auth-brand"><div class="mark"></div><span>AgriConnect</span></div>
16 <h1 style="font-size:1.5rem;">Welcome back</h1>
17 <p>Log in to your farmer or seller account.</p>
18 <div class="error-banner" id="error-banner"></div>
19 <form id="login-form">
20 <div class="field">
21 <label for="email">Email</label>
22 <input type="email" id="email" required autocomplete="email">
23 </div>
24 <div class="field">
25 <label for="password">Password</label>
26 <input type="password" id="password" required autocomplete="current-password">
27 </div>
28 <button type="submit" class="btn btn-block" id="submit-btn">Log in</button>
29 </form>
30 </div>
31 </div>
32 </body>
33 </html>
```



```
1 // Sets up the SQLite database (file-based, zero external server needed).
2 // Schema is plain SQL, so porting to MySQL/PostgreSQL later only means
3 // swapping the driver in this file - every route uses plain SQL strings.
4
5 const path = require("path");
6 const fs = require("fs");
7 const Database = require("better-sqlite3");
8
9 const DATA_DIR = path.join(__dirname, "..", "data");
10 if (!fs.existsSync(DATA_DIR)) fs.mkdirSync(DATA_DIR, { recursive: true });
11
12 const db = new Database(path.join(DATA_DIR, "agriconnect.db"));
13 db.pragma("journal_mode = WAL");
14 db.pragma("foreign_keys = ON");
15
16 db.exec(`
17 CREATE TABLE IF NOT EXISTS users (
18   id INTEGER PRIMARY KEY AUTOINCREMENT,
19   name TEXT NOT NULL,
20   email TEXT NOT NULL UNIQUE,
21   password_hash TEXT NOT NULL,
22   role TEXT NOT NULL CHECK(role IN ('farmer','seller')),
23   phone TEXT,
24   location TEXT,
25   created_at TEXT DEFAULT (datetime('now'))
26 );
27
28 CREATE TABLE IF NOT EXISTS crops (
29   id INTEGER PRIMARY KEY AUTOINCREMENT,
30   name TEXT NOT NULL UNIQUE,
31   category TEXT
32 );`);
```